

Worksheets for Temporal 3D Convolutional Neural Network for Predicting Radiation Pneumonitis within 90 days



AALBORG UNIVERSITY
DENMARK

GROUP HST-25-ST-8-8404

BIOMEDICAL ENGINEERING AND INFORMATICS MSC 2. SEMESTER 2025

AALBORG UNIVERSITY

Contents

1	Pre-processing	2
1.1	Intensity normalization	2
1.2	Resampling	2
1.3	Lung segmentation	3
1.4	Data augmentation	5
2	Model Design	7
2.1	Overview	7
2.2	ResNet-50	7
2.3	CNN for analyzing feature maps between scans	8
2.4	Post networks	9
3	Experimental protocols	15
3.1	Test training: 1	15
3.2	Test training: 3	17
3.3	Test training: 4	18
3.4	Test training: 5	19
3.5	Test training: 6	20
3.6	Test training: 7	21
4	Search protocol	22
	Bibliography	24

Appendix

1. Pre-processing

1.1 Intensity normalization

Medical images such as computed tomography (CT) scans are often affected by noise and artifacts. Specifically, CT has poisson noise, caused by the low photon counts in the detectors. Image normalization can help restore texture that has been lost by noise.[1]

The most common normalization methods used for Convolutional Neural Networks (CNN) are Min-Max normalization and Z-score normalization. Min-max normalization is simple, and can be described by the following equation 1.1:

$$I_{normalised} = \frac{I - I_{min}}{I_{max} - I_{min}} \quad (1.1)$$

In equation 1.1, I represents the image pixel intensity, I_{min} represents the minimum pixel intensity, and I_{max} is the maximum pixel intensity in the image.

In contrast, Z-score normalization takes into account both the mean and standard deviation of the pixel intensity distribution. The Z-score normalization can be described by equation 1.2:

$$I_{normalised} = \frac{I - mean}{std} \quad (1.2)$$

Since Z-score normalization gives values centered around 0, negative output values can be possible. The Z-score normalization was performed as part of pre-processing.[2]

1.2 Resampling

To make the CT scans consistent and therefore more suitable for use in a CNN, isometric resampling was performed on our data using the SimpleITK Python library. Our data consists of two different groups of CT scans, baseline scans taken before radiation therapy, and 1 and 4 month follow-up scans. As the follow-up scans and baseline scans were taken using two different CT machines, they have different voxel spacings in the axial and longitudinal directions. Although each group is internally consistent, the difference in spacings between them could potentially create difficulties when comparing data across groups. These differences affect the apparent size and shapes of anatomical structures or abnormalities found in the lungs, which could create problems for a CNN and possibly introduce

unwanted inconsistencies in the learned features. This is especially important when working with a 3D CNN, which relies on the spatial relationships between intensity values.

To address this problem, all scans were resampled to have a uniform voxel dimension of $1 \times 1 \times 1 \text{ mm}^3$. This is effectively done by defining a new grid of voxel centers, which is regularly spaced by 1 mm in physical space, and estimating the intensity value at each grid point. Since coordinates of these new points rarely match up with coordinates of points in the grid defined by the original scan, interpolation is needed. In this case, linear interpolation was used. Linear interpolation estimates values by averaging nearby values, weighted by the distance from the new point to the 8 nearby points in the old grid. This method is efficient and provides good accuracy for smooth data such as image data.[3] This process of defining grids and estimating values by interpolating from the grid defined by the original images can be described by the following formula:

$$f(\mathbf{x}) = \sum_{\mathbf{k} \in \mathbb{Z}^3} f_{\mathbf{k}} \varphi_{\text{int}}(\mathbf{x} - \mathbf{k})$$

where $f_{\mathbf{k}}$ are the original voxel values, $\mathbf{x}$ is a position in space, and φ_{int} is the interpolation function. In linear interpolation, this function is nonzero only for nearby points, which keeps computation local and thus more efficient. [3]

The original origin (coordinates for the bottom left voxel) was kept unchanged during resampling. This ensures that the original and resampled scans stay aligned in physical space.

In addition to isometric resampling, all volumes were resized to a fixed dimension of $224 \times 224 \times 224$, which is the expected input size for the 3D ResNet-50 model.[4, 5] This approach was chosen over simpler methods such as cropping, as the majority of segmented lungs exceeded 224 voxels in at least one spatial dimension. To preserve the apparent anatomical size, volumes were padded with their respective slope-intercept value to ensure cubic dimensions. This resampling was performed similarly to the isometric resampling, using SimpleITK and linear interpolation, with the key difference being the use of a fixed target size rather than a target spacing. Although, the result of this resampling lead to a different spacing than $1 \times 1 \times 1 \text{ mm}^3$, the spacing across all scans remained identical due to the identical cubic size of all scans.

1.3 Lung segmentation

The lungs were segmented using a lung segmentation model U-net(R231) by Hofmanninger et al.[6]. The model is public available and based on standard U-net architecture. The model is trained on

various of CT-scans of the chest including lungs with a broad range of pathologies and is able to include high-density abnormalities such as tumors and pleural effusions within the lung masks. The U-net(R231) model showed a Dice Similarity Coefficient (DSC) of 0.98 ± 0.03 in the study.[6] The U-net (R231) model uses input images with pixel values expressed in Hounsfield Units (HU) therefore the input images were converted to HU. The raw pixel values were converted to HU using the following formula:

$$HU = (pixelvalues * slope) + intercept$$

The slope and intercept values were extracted from the metadata of each CT scan.

1.3.1 Alignment

Alignment of the baseline and corresponding follow-up scan were performed, to achieve anatomical consistency across scans at two time points. To ensure this, rigid transformation matrices were computed to align follow-up scans with their corresponding baseline scans using anatomical landmarks. To avoid misalignment due to differences in inspiration levels across scans, it was important to select landmarks that were not affected by breathing. To this end, three vertebrae were chosen to serve as stable landmarks across scans, as the spine’s position in the upper body is relatively rigid and unaffected by breathing.

As it was initially unknown which three vertebrae would provide the best alignment, a segmentation of all thoracic vertebrae was performed. This was achieved by running inference on the entire dataset using the open-source segmentation model TotalSegmentator.[7, 8] For each scan, the segmentation masks of all thoracic vertebrae were used to calculate both the segmented volume and the coordinates of the center of mass in voxel space. These values were then saved to a CSV file along with their corresponding scan ID. To determine which vertebrae would result in the most accurate alignment, the saved values were analyzed in Python using Pandas. First, vertebrae that were not found by TotalSegmentator across all scans were excluded. Then, the volume of each segmentation was used as a proxy for segmentation quality. The three vertebrae with the highest average and minimum volume were selected, resulting in the use of T4, T8, and T10 as stable landmarks for alignment.

Based on the computed coordinates for each scan for a given patient, a rigid transformation matrix was calculated, consisting of a translation vector and a rotation matrix. The transformation matrix was computed using singular value decomposition (SVD), ensuring accurate alignment without scaling or deformation. This matrix was applied to the follow-up CT volume using the SimpleITK Python library, with linear interpolation to minimize artifacts.

1.4 Data augmentation

Preventing overfitting is a common challenge when training deep learning models. Overfitting refers to the phenomenon where a model struggles to learn generalizable features due to a lack of variation in the training data. Instead of identifying patterns that apply in the general case, the model ends up memorizing specific details of the training data, leading it to perform well on that training data but poorly on new, unseen data. This occurs especially when training datasets are small or limited. [9, 10] This is often the case in medical imaging due to the invasive nature of some imaging modalities, the prolonged imaging times, and the limited number of experts who have the needed expertise to label the images.[11] One way to accommodate this problem is via data augmentation. Data augmentation refers to the process of altering a dataset such that sample distributions more resembles the true distribution. [12] This can be achieved through various transformations applied to the dataset, such as geometric augmentations (e.g., flipping, rotation, translation, cropping), noise injection, sharpness, and brightness or contrast adjustments.[10] When performing data augmentation, it is important that the applied transformations are label-preserving, meaning the label of a given sample remains valid post-transformation. Whether a transformation is label-preserving is task-specific, for example, flipping a lung CT around its x-axis can be label-preserving for feature extraction, as lungs are mostly symmetrical, but might not be, for segmentation of the left and/or right lung. [10]

The augmentation used in this study were rotation, flipping, noise injection and sharpness. Rotation is applied as part of data augmentation to improve the model’s ability to handle variations in image orientation. The image is rotated with ± 10 degrees around the z-axis. Flipping is one of the most common augmentations techniques. Flipping alone represents a simple transformation but combined with other augmentation such as rotation, the model is exposed to a wider variety of changes in both orientation and spatial positioning. The images were flipped along the y-axis. In this study, flipping was applied both independently and in combination with rotation. For the noise injection, random adding of Gaussian noise is applied where the intensities are randomly modified.[13] The amount of Gaussian noise applied to a given scan was controlled by the standard deviation defined to be 0-0.3%, and the probability of Gaussian noise being applied to a given scan were set to 25%. The standard deviation controls the intensity of the noise by determining how much the pixel values are altered. The Gaussian noise was applied with the following formula:

$$I_{\text{noisy}} = I + \mathcal{N}(0, \sigma^2) \quad (1.3)$$

where I is the original image, I_{noisy} is a noisy version of I , $\mathcal{N}$ is the Gaussian noise, $\sigma^2 \in (0, 0.03)$ controls the intensity of the Gaussian noise applied.

The inclusion of different noise intensities enables the model to adapt more effectively to real-world data variability. Random adjusting sharpness is applied to enhance detail representation of objects of interest and improve model generalization. [14] By using sharpness as part of data augmentation, it helps improve the model’s robustness to images captured by different machines as the baseline and follow-up scans in this study. Adjusting the sharpness involves modifying the image’s edge definition and contrast. The amount of sharpness adjustment applied to a given scan was controlled by the alpha value defined to be 0–0.05, and the probability of sharpness adjustment applied to a given scan was set to 25%. The alpha parameter controls the strength of the sharpness adjustment. The sharpness was applied with the following formula:

$$I_{\text{adjusted}} = I + \alpha \cdot (I - I_{\text{blurred}}) \quad (1.4)$$

where I is the original image, I_{blurred} is a blurred version of I , $\alpha \in (0, 0.05)$ controls the strength of the sharpness adjustment.

2. Model Design

2.1 Overview

The overall model architecture was designed as shown in figure 2.1. The first part of the model consists of a 3D ResNet-50 network, pretrained on medical images. This network is implemented to extract features from baseline and follow-up scans in three different layers in the CNN. Afterwards, each resulting feature map are concatenated and processed further by another CNN trained to predict whether the patient will develop RP within the following 90 days. The concatenation is further explained in section 2.2.1.

To identify the most suitable network architecture, four variants were tried out: *CNN_2_Blocks*, consisting of three convolutional blocks; *CNN_4_Blocks*, consisting of five convolutional blocks; *CNN_6_Blocks*, comprising seven convolutional blocks; and *FC*, composed of four fully connected layers. The primary architectural differences between these models are detailed in section 2.4.

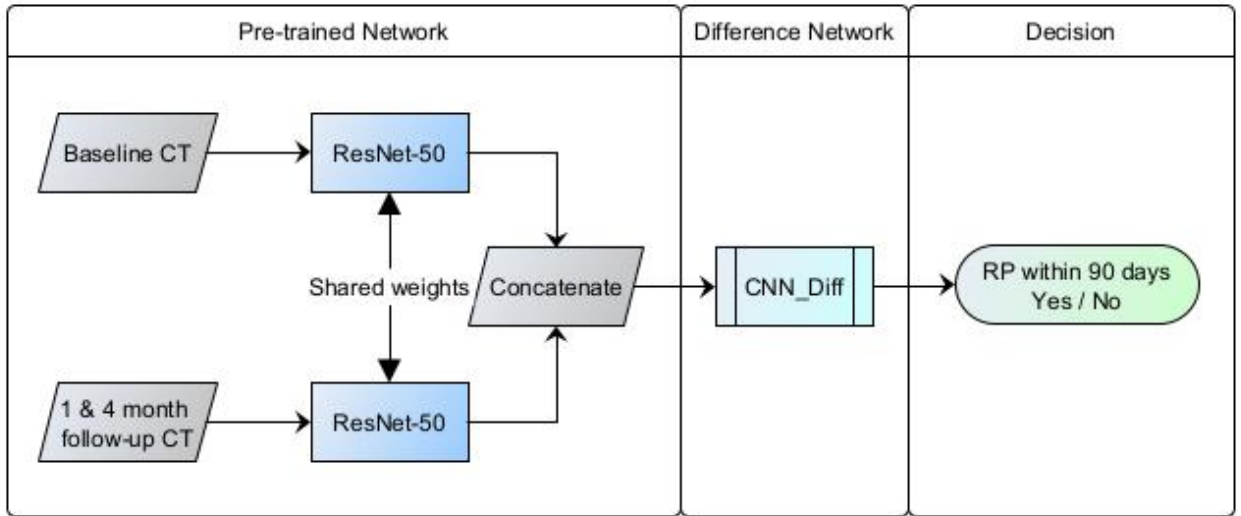


Figure 2.1: An overview of the overall model design. The model uses ResNet-50 to extract features from CT scans at two time points. These features are concatenated and passed through a CNN, which outputs whether the patient develops RP within the following 90 days.

2.2 ResNet-50

The CNN used for feature extraction were chosen to be a 3D ResNet-50 model from MONAI, pretrained on a large dataset of medical images, consisting of different imaging modalities and capturing different organs and areas of the body.[11] The choice of implementing a pre-trained model was based

on the results of Chen et al. [11], which showed that using models pre-trained on medical images converge faster and can improve accuracy in comparison to models pre-trained on more neutral datasets or models trained from scratch.

2.2.1 Concatenating the output

As mentioned, the outputs of the ResNet-50 network were concatenated, resulting in large feature maps. The feature maps were extracted from three increasingly detailed layers of ResNet: layer 2, 3, and 4. For each scan pair consisting of baseline and follow-up, the feature maps from each layer were first concatenated along the channel dimension, capturing temporal changes between baseline and follow-up. Since the spatial dimensions of each layer differed, all concatenated feature maps were resampled to match the resolution of layer 3: (1, 1024, 28, 28, 28). Finally, the resized feature maps from all three layers were concatenated along the channel dimension, resulting in a single tensor of shape (1, 7168, 28, 28, 28).

2.3 CNN for analyzing feature maps between scans

A study by Li et al. [15], demonstrated the ability to compute differences between images with a convolutional Siamese neural network to assess disease progression in retinopathy of prematurity and knee osteoarthritis. By comparing images from different time points, their model predicted disease severity on a continuous scale. Euclidean distance between images correlated with progression magnitude ($\rho = 0.74$ for ROP; $\rho = 0.55$ for osteoarthritis), and the model achieved an AUC of 0.90 in a change detection task. The study demonstrates the potential of using CNN to learn about disease progression.

Additionally, Szejniuk et al. [16] demonstrated that post-treatment follow-up CT scans can help predict RP onset. In a study of 41 non-small cell lung cancer patients receiving high-dose thoracic radiation therapy, they introduced a novel grading scale (RGS) for radiation-induced lung injury, based on radiologic findings. Higher RGS scores (2–3) were significantly associated with earlier and more severe RP, with median onset at 42 days versus 104 days for scores 0–1.

Based on these findings, we hypothesized that using baseline and 1- or 4-month follow-up CT scans enables the model to learn meaningful features relating to disease progression. This allows the prediction of RP development within 90 days of a given CT-scan of the lungs.

Even though Li et al. [15] showed promising results in determining disease progression using siamese network, it was chosen not to use a siamese network. This is due to this study not choosing to compute difference between scans, but rather predicting using another CNN to analyze the concatenated feature

maps of baseline and follow-up scans. The methods Li et al. [15] used, was not utilized, since calculating a scalar distance representing similarity between two images would not provide any information about the specific changes that occurred. As a result, it would be difficult to determine whether this distance is disease-related or caused by other factors.[17] The specific choice of the structure of the post CNN, was based on training with different networks which all outputs a binary classification, described in section 2.4.

2.3.1 Loss function and optimizer

In this study, binary cross-entropy was selected as the loss function, as it is widely adopted in the literature for binary classification tasks and has demonstrated robust performance in similar contexts.[18] Given that the current study involves a binary classification problem within a neural network framework, binary cross-entropy is an appropriate choice.

In this study, the Adam optimizer has been chosen, due to it being widely used in literature, and have proven to have good results.[19]

2.4 Post networks

The tested models will be described in the following section. To gain a better understanding of which architectural choices lead to better performance on the data, two types of neural networks have been investigated: three different CNN models and a simpler approach with a fully connected network. The goal has been to explore how changes in network depth, width of layers, and downscaling of features and dimensionality affect the performance.

All three CNN architectures were based on the VGG-16 architecture by Simonyan et al. [20], demonstrating that deeper networks yield higher classification accuracy. However, this was based on training with large-scale dataset rather than the dataset available in this study. Given the limited dataset, it was chosen to implement a reduced depth network architecture, though it remains inspired of the principles of the VGG architecture. Due to the input dimension of 28x28x28 instead of 224x224 as in the VGG net, max pool was only performed twice.

2.4.1 CNN with 4 convolutional blocks

The model follows a sequential block-wise design, consisting of five blocks: Four 3D convolutional blocks and one classifier block. Similar to the VGG architecture, a kernel size of 3x3x3 for all convolutional layers, max pooling layers with a stride of 2, and lastly, three fully connected layers were implemented. Furthermore, the first two blocks include two convolutional layers, and the following convolutional blocks include three convolutional layers. For preserving the dimensions throughout

the network, padding was set to 1 in the convolutional layers. The channels were reduced in each convolutional block, and the spatial dimensionality was reduced by max pooling at the end of blocks two and three, as seen in figure 2.2.

Block 1: The first block receives the high-dimensional input data with 7168 channels and dimension size at 28x28x28. This block contains two 3D convolutional layers. The first convolution reduces the number of channels to 2368, followed by batch normalization to stabilize the training, and a ReLU activation function to introduce non-linearity. The second convolution maintains the number of channels and the spatial resolution. The purpose of this block is to reduce the input features while preserving the spatial dimensions.

Block 2: The second block performs channel reduction and spatial downsampling. It consists of two 3D convolutional layers, each followed by batch normalization and a ReLU activation function. After, a 3D max pool was implemented to reduce the spatial dimensions. The purpose of this block is to compress channels and the spatial information from the input shape of (1,2368, 24, 24, 24) to the output shape (1, 768, 12, 12, 12).

Block 3: The third block is structurally very similar to block 2, it consists of three 3D convolutional layers. The first convolution reduces the channels from 768 to 256, the additional convolutions maintain the channel size. After each convolution, a batch normalization and a ReLU activation function are applied. At last, a max pool layer was implemented to reduce the spatial resolution again, resulting in an output shape (1, 256, 6, 6, 6).

Block 4: The fourth block contains three 3D convolutional layers, each with 64 channels, followed by a batch normalization and a ReLU activation function. This block maintains the channel size and the spatial resolution, although it focuses on refining the feature maps before flattening and classification.

Block 5 Classifier: The final block of the network is a classifier which consists of three fully connected layers. First, the output from block 4 is flattened into a 1D vector of 21,952 elements. The vector was passed through three fully connected layers, each followed by a ReLU activation function. The fully connected layers project the high-dimensional input down to 5488 elements, then 512 elements, and lastly into 2 output classes for the binary classification.

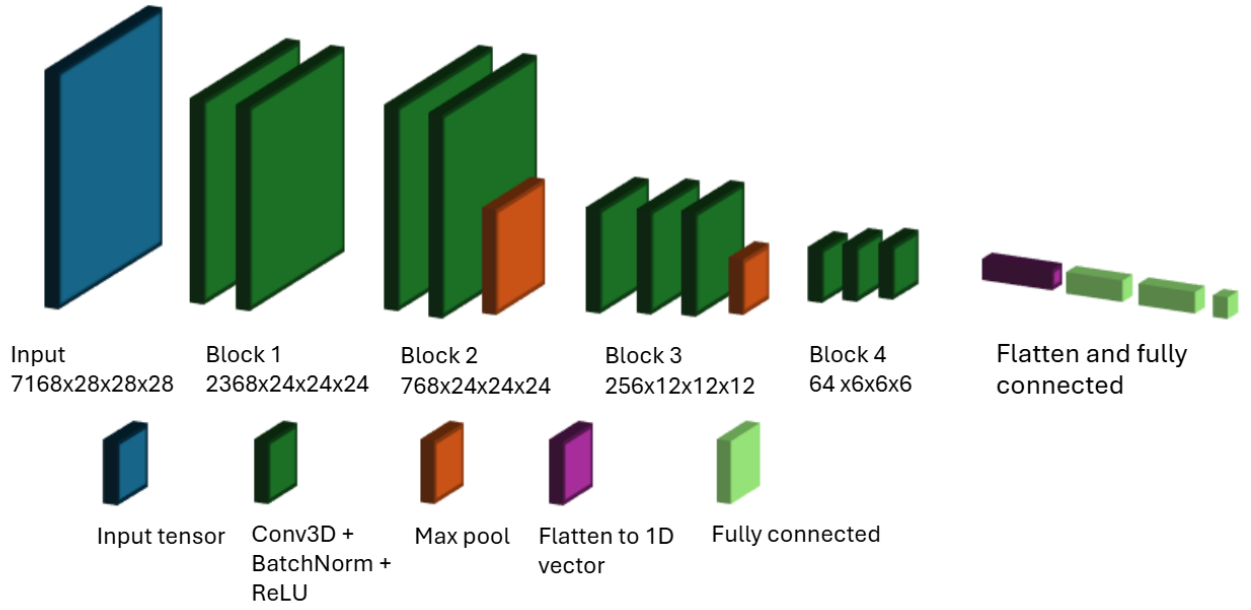


Figure 2.2: Model overview of the CNN with four convolutional and one fully connected block. The model consists of four similarly structured convolutional blocks, however block 2 and block 3 also includes max pooling. Lastly, a fully connected classifier block produces the output.

It is expected that this network is deep enough to learn meaningful representations, which could indicate whether convolutional processing is sufficient for the problem. Furthermore, the model consist of a few layers, which are expected to reduce the chances for overfitting due to the few ReLU activation functions. On the other hand, the model may not contain enough layers to learn the potentially very complex patterns in the data.

2.4.2 CNN with 6 convolutional blocks

This CNN halves the amount of features in each layer. Simultaneously, the dimensions is halved in every secondary layer. The network consists of six convolutional blocks and one classifier block. The architecture of this model can be seen in figure 2.3.

Block 1: The purpose of this block is to reduce features. It consists of a 3D convolutional layer which reduces features from 7168 to 3584, followed by a batch normalization and a ReLU layer. These layers are then repeated once in the same order, without reducing the features further.

Block 2: This block consists of further reduction of features, through a 3D convolutional layer, which reduces from 3584 to 1792. This is structured as block 1. Lastly, this is followed by a 3D max pooling layer, which halves the dimensions of the volumes from (24, 24, 24) to (12, 12, 12).

Block 3: This block is structured as an extended block 1 with three convolutional 3D layers, three batch normalization layers, and three ReLU layers. This way the features are reduced from 1792 to 896.

Block 4: The purpose of this block is to reduce features. This block is structured in the exact same way as block 3, reducing features from 896 to 448.

Block 5: The purpose of this block is to reduce features further, and reduce dimensionality. In the same structure as layer 3 and 4, the features are reduced from 448 to 224. Subsequently, a 3D max pooling layer reduces dimensions of the scan from (12, 12, 12) to (6, 6, 6).

Block 6: Serves as the last convolutional block, which reduces the features from 224 to 112, through the same structure as block 3 and 4.

Block 7 classifier: This is a classifier block, which consists of three fully connected layers. First a flattening layer to flatten the (112, 6, 6, 6) into 1D vector consisting of 38.416 features, which the fully connected layers takes as input. Then a fully connected linear layer which compresses the 38.416 features down to 512 neurons. Subsequently, a ReLU activation layer to add non linearity. Lastly, a final fully connected linear layer to give output (512, 2), to map the 512 features to the amount of classes the output can be predicted into.

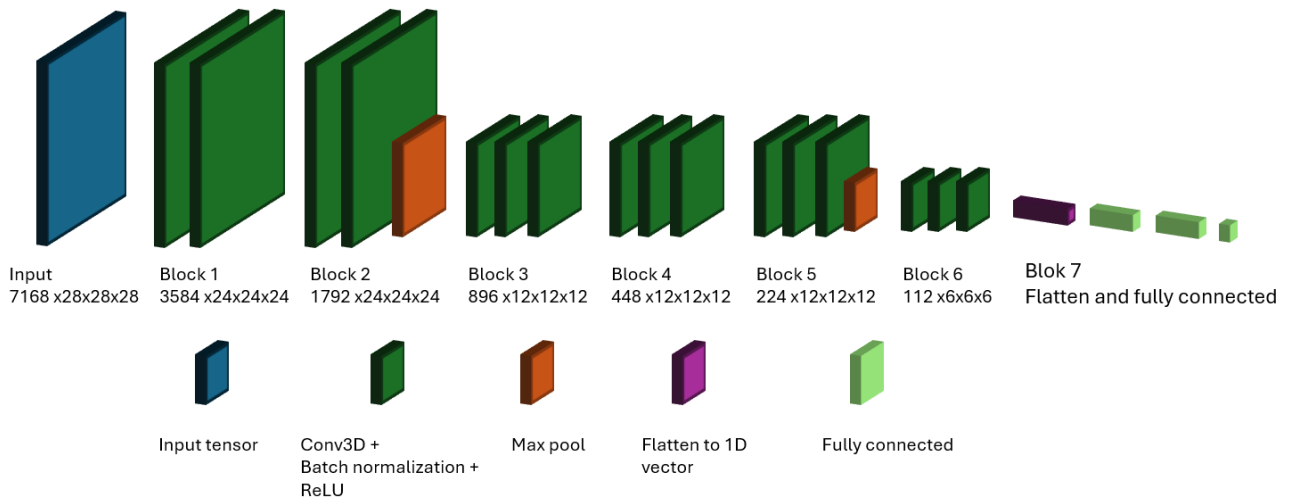


Figure 2.3: Model overview of the CNN with six convolutional and one fully connected block. The model consists of six similarly structured convolutional blocks, however block 2 and 5 also includes max pooling. Lastly, a fully connected classifier block produces the output.

The model is expected to be able to learn complex patterns in the data, due to the amount of layers. However, it can also be overly complex for the amount of data that the dataset consists of.

2.4.3 CNN with 2 convolutional blocks

This network architecture was developed to explore the performance of a minimalistic CNN composed of two convolutional blocks. The primary objective was to design a model simple enough to reduce the risk of overfitting, when keeping the limited size of the dataset in mind. The architecture can be

visualized in figure 2.4.

Block 1: The first block consists of two convolutions, the first convolution reduces the features from 7168 to 1024 and keeps the dimensionality to $24 \times 24 \times 24$, and is followed by a batch normalization and a ReLU activation function. The second convolution maintains both the features and the dimension size. At the end of the block, a max pooling is implemented, which halves the dimension, and results in an output shape of $(1, 1024, 12, 12, 12)$.

Block 2: The second convolutional block includes two convolutional layers, the first layer reduces the features once more from 1024 to 128 in the first convolution, and includes further more a batch normalization and a max pool which reduces the dimensionality to an output shape of $(1, 128, 6, 6, 6)$.

Block 3 classifier: Serves as the classifier block. In this block the feature map is flattened to a 1D vector of 27648 elements. The vector is comprised in three fully connected layers from 27648 to 4096 to 512, and lastly, into 2 output classes for the binary classification.

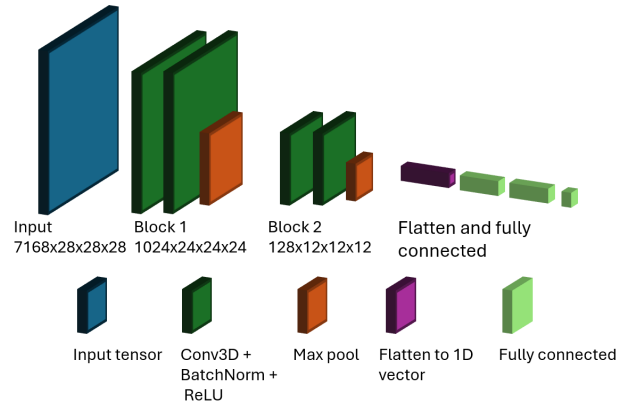


Figure 2.4: Model overview of the CNN with two convolutional and one fully connected block. The model consists of two similarly structured convolutional blocks where both blocks include max pooling. Lastly, a fully connected classifier block produces the output.

The motivation behind developing this model was to make a model that was so simple, that overfitting should not be a problem, due to the small amount of data.

2.4.4 Fully connected network

This model was designed with a hypothesis that the ResNet-50 output alone would be enough to classify whether a patient would develop RP within 90 days. Thus, a fully connected block would be enough. The architecture of the model can be seen in figure 2.5. This model consists purely of

fully connected layers. Before the actual fully connected block, an adaptive average pooling layer is implemented, in which the features are pooled to one dimension.

FC Block: This block begins with a flattening layer that reshapes the feature maps from a shape of (7168, 1, 1, 1) to (7168). This is followed by a fully connected layer that reduces the dimensionality to 1024, after which a ReLU activation function is applied to introduce non-linearity. A second linear layer further reduces the feature size to 512, followed by another ReLU activation. This pattern continues with a third linear layer reducing the size to 128, again followed by a ReLU activation. Finally, a fourth linear layer maps the output to the desired output size of 2.

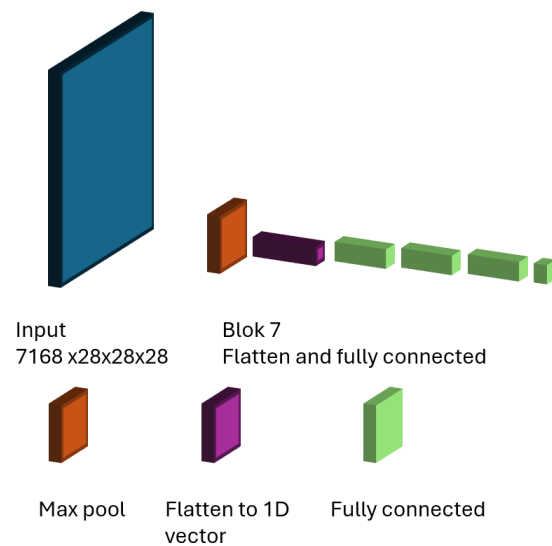


Figure 2.5: Model overview of the FC network. A feature map is passed as input, and flattened into a 1D vector. Afterwards the amount of features are reduced from 7168 to 128, through 4 linear layers with a ReLU layer between each layer.

The expected outcome is a faster and more stable model convergence, which may help indicate whether convolutional blocks are necessary.

3. Experimental protocols

3.1 Test training: 1

Date: 14 May 2025

Experiment ID: Test_Run_001

CNN Architecture				
CNN_7_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	2	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
$1 \cdot 10^{-4}$	3	20	Adam	CrossEntropy
Notes/Observation				
This is a test, to see whether the model actually works and can run. Thus, this is not an actual training. Therefore, no performance metrics was printed or plotted.				

3.1.1 Test training: 2

Date: 15 May 2025

Experiment ID: Test_Run_002

CNN Architecture				
CNN_5_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
$1 \cdot 10^{-4}$	3	100	Adam	CrossEntropy
Notes/Observation				
According to the highly skewed class distribution, torch WeightedRandomSampler was implemented in the training. As the CNN_7_Block had an unacceptable training time according to the time frame of this study. As well as considerations that the model was too deep according to the dataset, a reduced-depth model was implemented, the CNN_5_Block.				

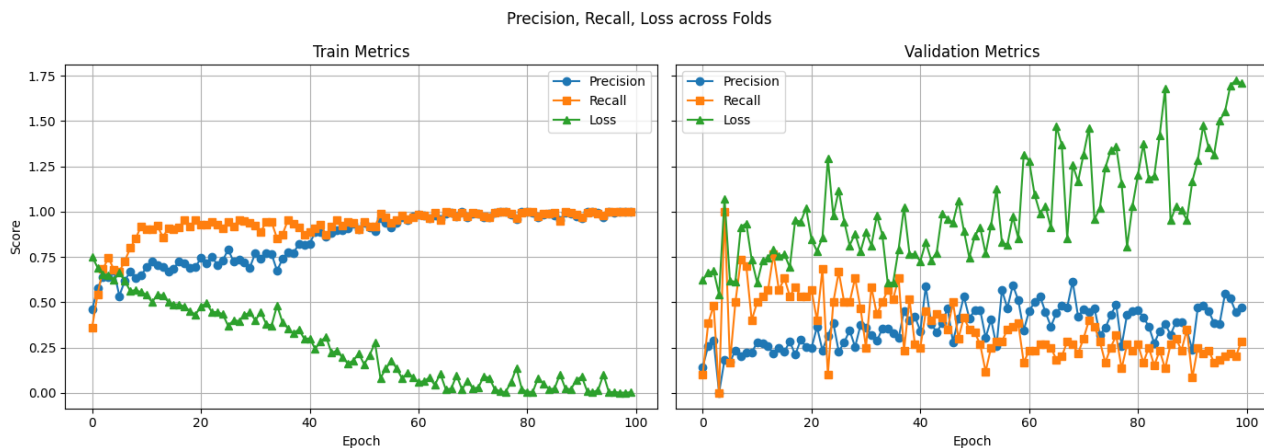


Figure 3.1: Performance metrics for test training 2, Test_run.002

3.2 Test training: 3

Date: 15 May 2025

Experiment ID: Test_Run_003

CNN Architecture				
CNN_5_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
$1 \cdot 10^{-4}$	3	100	Adam	CrossEntropy
Notes/Observation				
After further investigation, the model's prediction in test_run_002, the WeigtedRandomSampler was reduced by 25% in class 1 for this training, because of the high amount of class 1 predictions in test_run_002. Due to the extreme variation in class distribution, the distribution focus is now mainly on distributing depending on class labeling and therefore not ensuring patient overlap between training and validation. This means that the same patient can possibly be represented in both training and validation because each patient can contribute to one or two datasets. This results in the same baseline for can be represented in both training and validation				

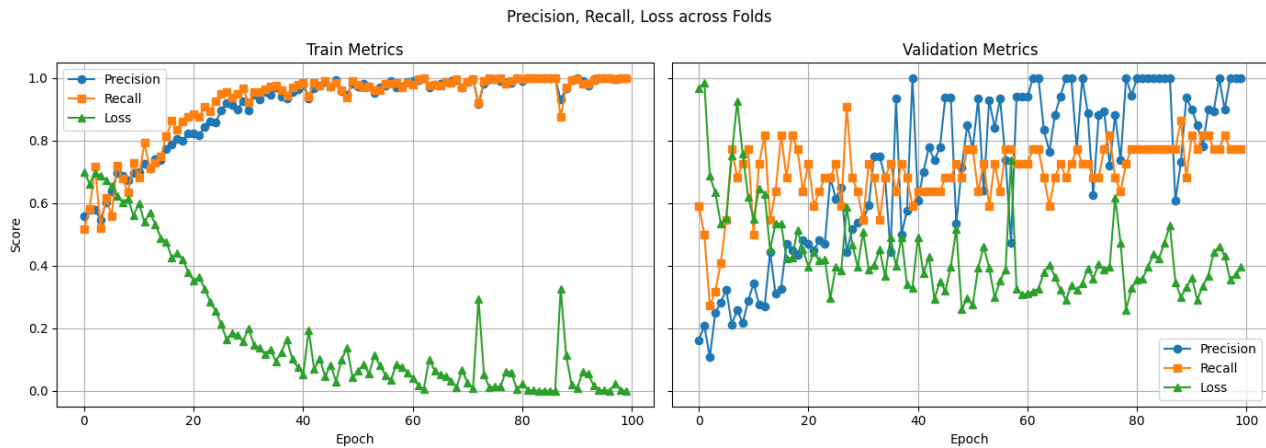


Figure 3.2: Performance metrics for test training 3, Test_run_003

3.3 Test training: 4

Date: 16 May 2025

Experiment ID: Test_Run.004

CNN Architecture				
CNN_5_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
None	3	100	Adam	CrossEntropy
Notes/Observation				
WeigtedRandomSampler is still implanted with reduced weights of 25% for class 1. From Test_run_003 we saw positive evaluation in the validation metrics, this could be due to the patient overlap, therefore, the distribution is changed back to distribute with focus on patients, resulting in no patient overlap between training and validation as in Test_run.002. The purpose for this run was to evaluate if it was the patient overlap that was the reason for the better evolving validation metrics during Test_run_003				



Figure 3.3: Performance metrics for test training 4, Test_run.004

3.4 Test training: 5

Date: 16 May 2025

Experiment ID: Test_Run.005

CNN Architecture				
CNN_5_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
None	3	100	Adam	CrossEntropy
Notes/Observation				
In this training WeigtedRandomSampler with reduced weights 50% and weights in CrossEntropy-Loss() was implanted. This was changed according to Test_run_004 where it was considered that the model rapidly overfitted due to the missing improvement in the validation metrics.				



Figure 3.4: Performance metrics for test training 5, Test_run.005

3.5 Test training: 6

Date: 17 May 2025

Experiment ID: Test_Run_006

CNN Architecture				
CNN_5_Block				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
None	3	100	Adam	CrossEntropy
Notes/Observation				
According the the missing improvement in the validation metrics in Test_run_004 and Test_run_005 dropout was introduced to try another way to prevent overfitting and force the model to learn less specific features. Therefore both the WeigtedRandomSampler and dropout was introduced. Dropouts was implemented in each block as following: Block 1: 0.1, Block 2: 0.1, Block 3: 0.2, Block 4: 0.3, Block 5: 0.3				

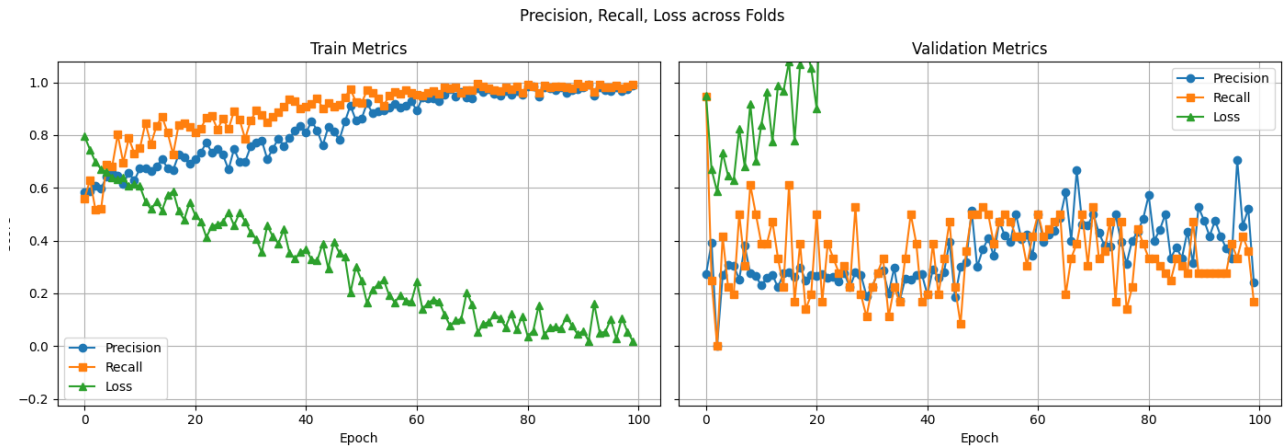


Figure 3.5: Performance metrics for test training 6, Test_run_006

3.6 Test training: 7

Date: 18 May 2025

Experiment ID: Test_Run.007

CNN Architecture				
FC				
Data Augmentation				
Add Gaussian noise		Adjust sharpness		
25%		25%		
Training Parameters				
Randomsampler	K-folds	Extracted Layers	Concatenation	Learning Rate
Yes	1	2, 4, 5	Yes	$1 \cdot 10^{-4}$
Weight Decay	Batch size	Epochs	Optimizer	Loss function
None	3	100	Adam	CrossEntropy
Notes/Observation				
Build on the earlier trainings with CNN architecture the group wants to investigate wether it is necessary to implement a CNN network or if a simpler fully connected network could perform as well. Therefore a fully connected network is introduced width WeigtedRandomSampler				

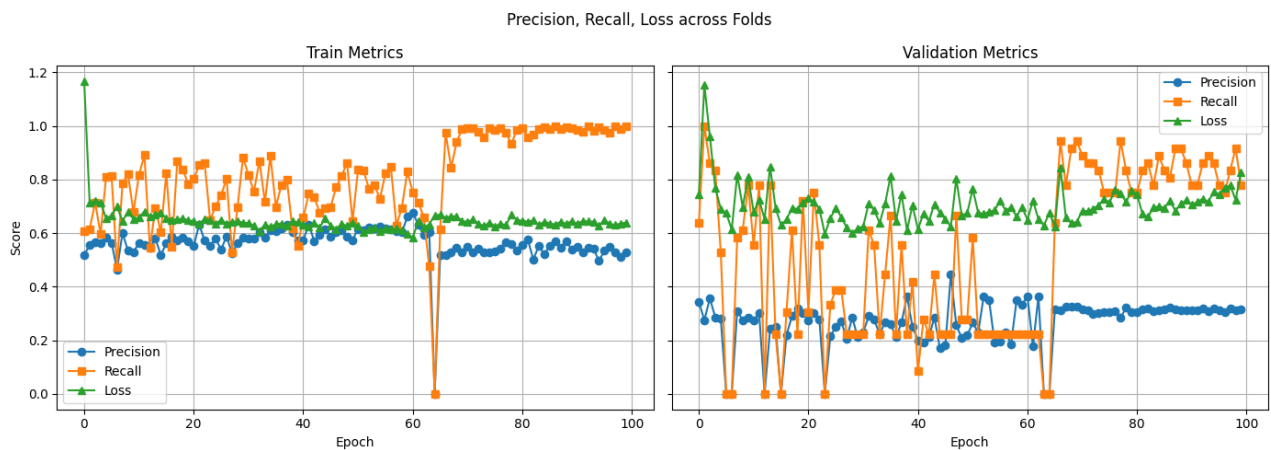


Figure 3.6: Performance metrics for test training 7, Test_run.007

4. Search protocol

Definition of research subject

We investigated whether previous studies have employed CT imaging to predict RP in patients with lung cancer. The research subject is based on the PICO model visualized in table 4.1.

P	I	C	O
Patient group	Intervention	Comparison	Outcome
Patients with lung cancer	CT and Machine Learning	Compare lung cancer patients with and without radiation induced pneumonitis.	Prediction of radiation induced pneumonitis.

Table 4.1: The PICO model used for the research subject.

Inclusion and exclusion criteria

In order to narrow the search and ensure relevance to the research subject, the inclusion and exclusion criteria are defined as seen in table 4.2.

Inclusion criteria	Exclusion criteria
English and nordic language Full-text Peer-reviewed	Published before 2010

Table 4.2: The inclusion and exclusion criteria for the search.

Aspects of the search

For the search, aspects are defined including relevant terms related to the research subject. The aspects are defined in table 4.3.

AND				
OR	Aspect 1	Aspect 2	Aspect 3	Aspect 4
	Radiation induced pneumonitis	CT	Machine Learning	Predict*
	Radiation pneumonitis	Computed Tomography	Deep Learning	Develop*
	Radiation induced lung injur*		Convolutional Neural Network	Detect*
	RILI		CNN	

Table 4.3: The list of the four search aspects.

Selection of sources

In table 4.4, the selected sources and the reason are defined to investigate the research subject.

Source	Reason for selection of source	Provider
PubMed	Contains medical and nature science publications.	AUB
Embase	Contains biomedical and pharmacological research.	AUB
Scopus	Broad database containing life sciences, social sciences, physics and health sciences.	AUB

Table 4.4: The listed sources used in the search.

PRISMA flow diagram

To summarize the articles, a PRISMA flow diagram is visualized in figure 4.1 where the exclusion process is emphasized. After the duplicate exclusion, the exclusion process is divided into three phases where article is excluded based on their title, abstract and full text.

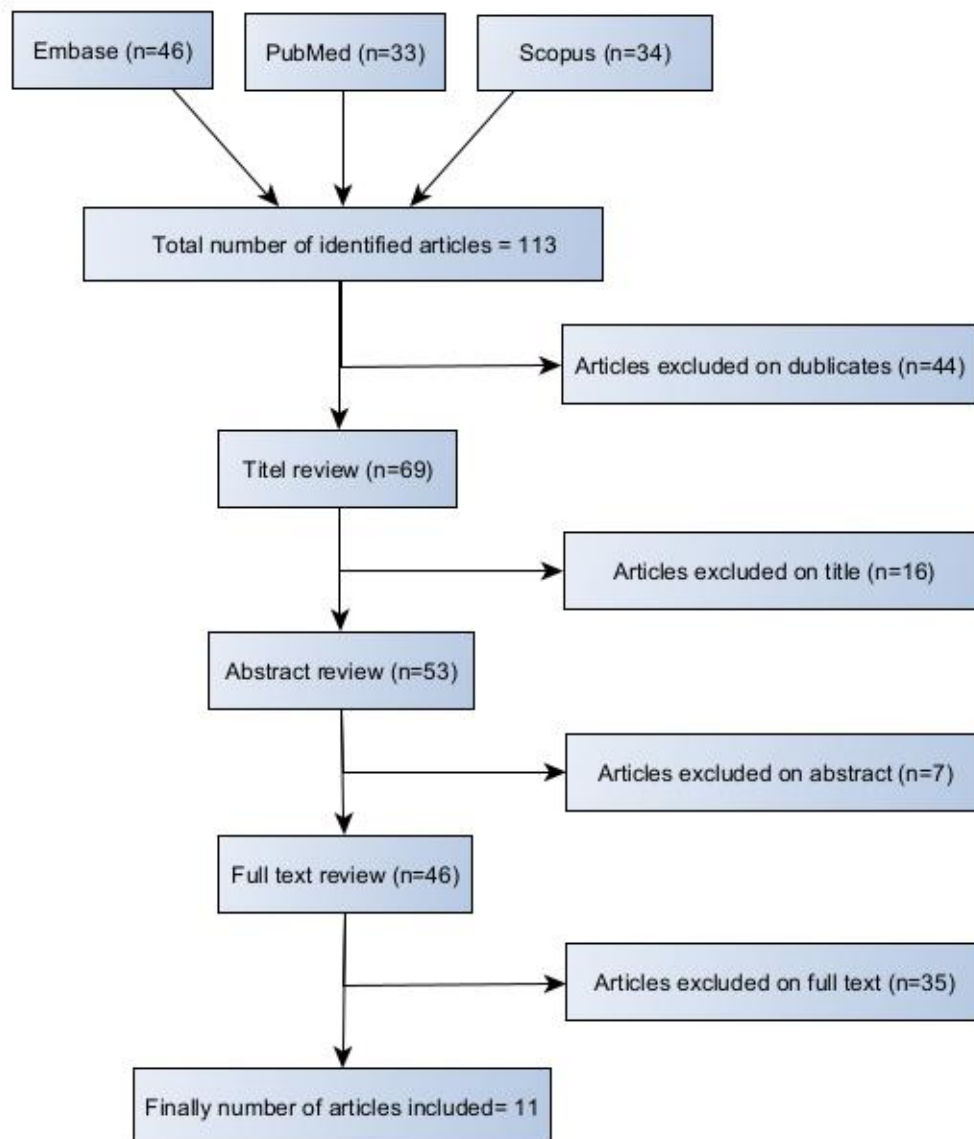


Figure 4.1: Visualization of the PRISMA of the search protocol with the number of excluded in the exclusion process.

Bibliography

- [1] T. Toyoda et al. “Application of machine-learning models to improve the image quality of photon-counting CT images”. In: *Journal of Instrumentation* 16.05 (May 2021), P05021. DOI: 10.1088/1748-0221/16/05/P05021.
- [2] Henderi Henderi, Tri Wahyuningsih, and Efana Rahwanto. “Comparison of Min-Max normalization and Z-Score Normalization in the K-nearest neighbor (kNN) Algorithm to Test the Accuracy of Types of Breast Cancer”. In: *International Journal of Informatics and Information Systems* 4.1 (2021), pp. 13–20. ISSN: 2579-7069. DOI: 10.47738/ijis.v4i1.73.
- [3] Philippe Thévenaz, Thierry Blu, and Michael Unser. “Interpolation and resampling”. In: *Handbook of Medical Imaging*. Last visited: 23/05-2025. Academic Press, 2000, pp. 393–420. URL: <https://books.google.dk/books?id=UHkkPBnhT-MC>.
- [4] Adelisa Sirco et al. “Liver tumour segmentation based on ResNet technique”. In: *2022 IEEE 12th International Conference on Control System, Computing and Engineering (ICCSCE)*. IEEE. 2022, pp. 203–208. DOI: 10.1109/ICCSCE54767.2022.9935636.
- [5] Shiyi Wang and Guang Yang. “A novel automated classification and segmentation for COVID-19 using 3D CT scans”. In: *2022 IEEE 5th International Conference on Image Processing Applications and Systems (IPAS)*. IEEE. 2022, pp. 1–5. DOI: 10.1109/IPAS55744.2022.10052819.
- [6] Johannes Hofmanninger et al. “Automatic lung segmentation in routine imaging is primarily a data diversity problem, not a methodology problem”. In: *European radiology experimental* 4 (2020), pp. 1–13. DOI: 10.1186/s41747-020-00173-2.
- [7] Tugba Akinci D’Antonoli et al. “TotalSegmentator MRI: Robust Sequence-independent Segmentation of Multiple Anatomic Structures in MRI”. In: *Radiology* 314.2 (2025), e241613. DOI: 10.1148/radiol.241613.
- [8] Fabian Isensee et al. “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation”. In: *Nature methods* 18.2 (2021), pp. 203–211. DOI: 10.1038/s41592-020-01008-z.
- [9] Gary Murphy and Raghubir Singh. “Comparative Analysis and Ensemble Enhancement of Leading CNN Architectures for Breast Cancer Classification”. In: *arXiv preprint arXiv:2410.03333* (2024). DOI: 10.48550/arXiv.2410.03333. arXiv: 2410.03333 [cs.CV].

- [10] Connor Shorten and Taghi M Khoshgoftaar. “A survey on image data augmentation for deep learning”. In: *Journal of big data* 6.1 (2019), pp. 1–48. DOI: 10.1186/s40537-019-0197-0.
- [11] Sihong Chen, Kai Ma, and Yefeng Zheng. “Med3D: Transfer Learning for 3D Medical Image Analysis”. In: *arXiv preprint arXiv:1904.00625* (2019). DOI: 10.48550/arXiv.1904.00625.
- [12] Ruqian Hao et al. “A comprehensive study of data augmentation strategies for prostate cancer detection in diffusion-weighted MRI using convolutional neural networks”. In: *Journal of Digital Imaging* 34 (2021), pp. 862–876. DOI: 10.1007/s10278-021-00478-7.
- [13] S. Harris, J. Smith, and A. Lee. “Advanced Imaging Techniques in Radiology”. In: *Journal of Medical Imaging* 45.3 (2022), pp. 123–130. DOI: 10.1111/1754-9485.13261.
- [14] Connor Shorten and Taghi M Khoshgoftaar. “A survey on image data augmentation for deep learning”. In: *Journal of Big Data* 6.1 (2019), pp. 1–48. DOI: 10.1186/s40537-019-0197-0.
- [15] Matthew D Li et al. “Siamese neural networks for continuous disease severity evaluation and change detection in medical imaging”. In: *NPJ digital medicine* 3.1 (2020), p. 48. DOI: 10.1038/s41746-020-0255-1.
- [16] Weronika Maria Szejniuk et al. “High-dose thoracic radiation therapy for non-small cell lung cancer: a novel grading scale of radiation-induced lung injury for symptomatic radiation pneumonitis”. In: *Radiation Oncology* 16 (2021), pp. 1–11. DOI: 10.1186/s13014-021-01857-8.
- [17] Tareque Rahman Ornob, Gourab Roy, and Enamul Hassan. “CovidExpert: A Triplet Siamese Neural Network framework for the detection of COVID-19”. In: *Informatics in Medicine Unlocked* 37 (2023), p. 101156. ISSN: 2352-9148. DOI: 10.1016/j.imu.2022.101156.
- [18] Usha Ruby, Vamsidhar Yendapalli, et al. “Binary cross entropy with deep learning technique for image classification”. In: *Int. J. Adv. Trends Comput. Sci. Eng* 9.10 (2020). DOI: 10.30534/ijatcse/2020/175942020.
- [19] Ochin Sharma. “Deep challenges associated with deep learning”. In: *2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon)*. IEEE. 2019, pp. 72–75. DOI: 10.1109/COMITCon.2019.8862453.
- [20] Karen Simonyan and Andrew Zisserman. *Very deep convolutional networks for large-scale image recognition*. 2014. DOI: 10.48550/arXiv.1409.1556.